Post Test

Sanskruti Shukla AIML C 22
VI Sem

Answer 1. Constructor called
Init called

Answer 2. Return new ResponseEntity<>(u,HttpStatus.OK);

Answer 4.

Answer 6. This filter checks the JWT token from the authorization header of the request.
If the token is valid, it authenticates the user and stores the authentication in Spring Security's Context.

The bug is that "Bearer " is 7 characters long, but the code starts from index 6, so part of "Bearer " remains in the token.

String token = header.substring(7);

Answer 7. @Query("SELECT p FROM Product p WHERE p.price < :price AND p.category = :category")

List<Product> findProductsByPriceAndCategory(

@Param("price") double price,

@Param("category") String category

);

Answer 8. The N+1 query problem happens when JPA first runs 1 query to fetch a list of parent objects, and then runs N extra queries to fetch related child objects one by one.

Example:
Suppose you fetch 10 students, and each student has a department.
JPA runs: 1 query to get all students & 10 more queries to get each student's department
So total = 11 queries.

The most common cause is lazy loading (FetchType.LAZY) on relationships.

Answer 9. SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

JPQL : @Query("SELECT MAX(e.salary) FROM Employee e WHERE e.salary < (SELECT MAX(e2.salary) FROM Employee e2)")

Answer 10: List<Student> findByCity(String city);

List<Student> findByNameAndCity(String name, String city);

List<Student> findByEmailContaining(String keyword);

long countByCity(String city);

Answer 11. N+1 Query problem.

The line causing it is: System.out.println(o.getItems().size());

This calls a separate query for items of each order because of lazy loading.

@Query("SELECT o FROM Order o JOIN FETCH o.items")

Answer 12. application.yml:

```
spring:
  application:
    name: product-service

eureka:
  client:
    service-url:
```

defaultZone: http://localhost:8761/eureka

Main class annotation: @EnableEurekaClient

Answer 13. A Circuit Breaker is a design pattern in microservices that stops repeated calls to a failing service. It is needed to prevent system failure, reduce unnecessary load, and provide fallback responses when another service is down.

Answer 14. An API Gateway is a single entry point for all client requests in a microservices architecture. It forwards requests to the appropriate microservice.

Benefits: 1. Centralized security, authentication, and routing.

2. Load balancing and easier management of multiple microservices.

application.yml:

```
Spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/api/orders/**

        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**
```

Answer 15: Service Discovery is a mechanism where microservices register themselves with a central server so other services can find them dynamically.

It is needed because IP addresses and ports may change frequently, especially in cloud or container environments. Hardcoding them makes the system difficult to maintain.

The tool used is Netflix Eureka.

RestTemplate Bean Configuration: @Bean

```
@LoadBalanced

public RestTemplate restTemplate() {

    return new RestTemplate();

}
```

@LoadBalanced annotation allows RestTemplate to resolve service names like:
http://PRODUCT-SERVICE/api/products

Answer 16. Benefits of splitting into microservices:

1. Each service can be developed and deployed independently.
2. Different services can scale separately based on load.
3. A failure in one service is less likely to bring down the entire application.

Challenges:

1. Increased complexity in communication between services.
2. More difficult monitoring, debugging, and deployment management.

Eureka + API Gateway working together:

Netflix Eureka acts as a Service Discovery server where all microservices (User, Order, Product) register themselves.

The API Gateway receives requests from the React frontend through a single URL and asks Eureka for the correct service location.
It then forwards the request to the appropriate microservice automatically without hardcoded IP addresses.

Answer 18. @Mock is a Mockito annotation used for unit testing without loading the Spring context. Use @Mock for simple unit tests. @MockBean is a Spring Boot testing annotation that creates a mock bean and places it inside the Spring application context. Use @MockBean when testing Spring Boot components with @SpringBootTest.

```
@SpringBootTest

public class UserServiceTest {


    @MockBean

    private UserRepository userRepository;


    @Autowired

    private UserService userService;

}
```

Answer 19. FROM eclipse-temurin:21-jre

```
WORKDIR /app


COPY target/app.jar app.jar


EXPOSE 8080


ENTRYPOINT ["java", "-jar", "app.jar"]
```

Answer 20. version: '3.9'

```
services:

  db:
    image: postgres:15
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password

  app:
    image: myapp:latest
    ports:
      - "8080:8080"
    depends_on:
      - db
```

Answer 22. useEffect() has no dependency array, so it runs after every render.

Since setUsers() causes a re-render, the API keeps getting called infinitely.